

Informe Consolidado QA

Sistema CRUD de Clientes EFAC

Analisis del proyecto, estrategia de pruebas, automatizacion y evidencias

Dato	Valor
Proyecto	EFAC QA Testing y Automatizacion
Tecnologia	ASP.NET Core .NET 9, Razor Pages, Web API, xUnit, Selenium
Repositorio	https://github.com/carlosgarciasalas-soporte/QaTestAuto.git
Rama / Commit	main / 3ce426e
Fecha de consolidacion	2026-05-11 12:42

Proposito del documento

Consolidar en un solo PDF la informacion relevante del README, la matriz de evidencias, el plan de automatizacion y el estado actual de pruebas automaticas del proyecto EFAC.

1. Resumen ejecutivo

EFAC es un sistema CRUD de clientes construido con ASP.NET Core .NET 9. Administra clientes bajo reglas de negocio asociadas a cumplimiento DIAN: normalizacion de NIT, calculo de digito de verificacion, control de duplicidad, validacion de mayoría de edad y diferenciacion entre persona natural y juridica.

Estado general

El proyecto cuenta con pruebas automatizadas de API y UI, script de ejecucion completo para Windows, reportes TRX, log persistente y documentacion de evidencias. La ultima validacion consolidada reporta 11 pruebas API superadas y 10 pruebas Selenium superadas.

Area	Resultado consolidado
Backend/API	Endpoints CRUD y calculo DV disponibles en /api/clientes.
Frontend	Interfaz Razor Pages con IDs estables para automatizacion Selenium.
Persistencia	Entity Framework Core InMemory para ejecucion academica y pruebas.
Automatizacion API	11 pruebas xUnit con WebApplicationFactory y FluentAssertions.
Automatizacion UI	10 pruebas Selenium con Chrome visible o headless.
Evidencias	Log QA, reportes TRX, guia de capturas y matriz de trazabilidad.

2. Alcance y fuentes consolidadas

Este informe integra la informacion operativa y tecnica necesaria para revisar el proyecto, analizar su cobertura de pruebas y ejecutar el ciclo QA automatizado.

Fuente	Contenido integrado
README.md	Descripcion, arquitectura, reglas, ejecucion, automatizacion y evidencias.
QA_Matriz_Evidencias_Pruebas.md	Trazabilidad, casos manuales, evidencias, campos y resultados esperados.
AUTOMATIZACION_PLAN.md	Etapas de automatizacion, criterios de avance y cobertura automatizada.
Codigo de pruebas	Casos API y Selenium implementados en los proyectos de prueba.
Scripts QA	Ejecucion automatica con run-qa-tests.bat y run-qa-tests.ps1.

3. Vision tecnica del proyecto

3.1 Arquitectura

Proyecto	Responsabilidad
Efac.Domain	Entidad Cliente, enums, excepciones y calculo DIAN modulo 11.
Efac.Application	DTOs, servicios de caso de uso, validaciones y contratos.
Efac.Infrastructure	DbContext, repositorio e inyeccion de dependencias de persistencia.
Efac.WebAPI	Controladores REST, Swagger, Razor Pages e interfaz web.
Efac.Tests.Api	Pruebas automatizadas de endpoints y reglas de negocio.
Efac.Tests.Selenium	Pruebas E2E de interfaz con navegador.
test-assets	Estructura para evidencias de ejecucion, capturas y reportes.

3.2 Endpoints disponibles

Metodo	Ruta	Uso
GET	/api/clientes	Listar clientes.
GET	/api/clientes/{id}	Consultar cliente por identificador.
GET	/api/clientes/calcular-dv/{nit}	Normalizar NIT y calcular DV.
POST	/api/clientes	Crear cliente natural o juridico.
PUT	/api/clientes/{id}	Actualizar cliente existente.
DELETE	/api/clientes/{id}	Eliminar cliente existente.

3.3 Reglas de negocio principales

- El NIT se normaliza antes de persistir, buscar o calcular el DV.
- El digito de verificacion se calcula con algoritmo DIAN modulo 11.
- No se permite registrar dos clientes con el mismo NIT.
- La persona natural requiere nombres, apellidos y fecha de nacimiento.
- La persona natural menor de edad debe ser rechazada.
- La persona juridica requiere razon social.
- Los campos no aplicables al tipo de persona no deben persistirse.

4. Estrategia QA

Nivel	Herramienta	Objetivo
Prueba manual API	Swagger	Validar contratos y payloads durante exploracion manual.
Prueba automatizada API	xUnit + WebApplicationFactory	Validar reglas HTTP, negocio y persistencia en memoria.
Prueba E2E UI	Selenium WebDriver	Simular flujos reales de usuario en navegador.
Evidencias	Logs, TRX y capturas	Sustentar ejecucion y resultado de pruebas.
Orquestacion	run-qa-tests.bat / .ps1	Ejecutar restore, build, API, UI, suite completa y reportes.

5. Cobertura automatizada

5.1 Pruebas API

ID	Caso	Resultado esperado
API-001	Listar clientes	200 OK con datos semilla.
API-002	Consultar cliente inexistente	404 Not Found.
API-003	Calcular DV con NIT formateado	200 OK, NIT normalizado y DV calculado.
API-004	Rechazar NIT invalido	400 Bad Request.
API-005	Crear persona natural valida	201 Created.

ID	Caso	Resultado esperado
API-006	Rechazar NIT duplicado	409 Conflict.
API-007	Rechazar menor de edad	400 Bad Request.
API-008	Rechazar natural sin fecha	400 Bad Request.
API-009	Rechazar juridica sin razon social	400 Bad Request.
API-010	Actualizar cliente	200 OK con datos actualizados.
API-011	Eliminar cliente	204 No Content y posterior 404.

5.2 Pruebas Selenium

ID	Caso	Controles o flujo validado
UI-001	Carga principal	Titulo, input-search y btn-new-client.
UI-002	Busqueda por NIT	input-search y tabla de clientes.
UI-003	Alternancia Natural/Juridica	Campos naturales y razon social.
UI-004	Calculo DV	input-nit, input-dv y llamada a API.
UI-005	Crear natural	Formulario completo y aparicion en tabla.
UI-006	Crear juridica	Tipo persona, razon social y guardado.
UI-007	NIT duplicado	form-validation-summary con error esperado.
UI-008	Menor de edad	fechaNacimiento y mensaje de error.
UI-009	Editar cliente	Accion Editar y persistencia del cambio.
UI-010	Eliminar cliente	Accion Eliminar, confirmacion y ausencia en tabla.

5.3 Campos validados

Campo	Validacion
tipoPersona	Diferencia Natural y Juridica; activa/oculta campos segun tipo.
nit	Normalizacion, busqueda, duplicidad y calculo de DV.
dv	Calculo DIAN modulo 11.
nombres / apellidos	Obligatorios y visibles para persona natural.
fechaNacimiento	Obligatoria para natural; valida mayoria de edad.
razonSocial	Obligatoria y visible para persona juridica.
email / telefono / direccion	Persistencia en creacion y actualizacion.
ciudadCodigoMunicipio	Persistencia del municipio.
responsabilidadFiscal	Persistencia del regimen fiscal seleccionado.

6. Flujo de ejecucion y evidencias

6.1 Ejecucion automatica

1. Abrir CMD o PowerShell en la raíz del proyecto.
2. Ejecutar `.\run-qa-tests.bat`.
3. Verificar el encabezado EFAC - EJECUCION QA AUTOMATICA.
4. Esperar restore, build, pruebas API, WebAPI local, Selenium y suite completa.
5. Tomar capturas del navegador y consola durante la ejecucion.
6. Revisar la carpeta indicada en Reportes TRX.
7. Presionar una tecla para cerrar la ventana al finalizar.

6.2 Archivos generados

Archivo	Evidencia
qa-execution.log	Registro completo de comandos, etapas y resultados.
api-tests.trx	Resultado formal de las 11 pruebas API.
selenium-tests.trx	Resultado formal de las 10 pruebas Selenium.
*.trx adicionales	Resultados de la suite completa.
webapi.log	Salida de la WebAPI durante el ciclo automatico.

6.3 Capturas recomendadas

- Inicio del script con el encabezado de ejecucion automatica.
- Compilacion correcta.
- Pruebas API con Superado: 11.
- Chrome visible llenando formularios, buscando, editando y eliminando clientes.
- Errores visibles por NIT duplicado y menor de edad.
- Pruebas Selenium con Superado: 10.
- Resumen final con Restore, Build, API, Selenium y Suite completa en OK.
- Carpeta generada en test-assets/evidence/reports.

7. Evidencia visible en GitHub

Tipo	Archivo	Evidencia
Codigo	Efac.Tests.Api/CientesApiTests.cs	Casos API y reglas de negocio.
Codigo	Efac.Tests.Selenium/Tests/CientesUiSmokeTests.cs	Flujos UI CRUD y validaciones negativas.
Codigo	Efac.Tests.Selenium/Pages/CientesPage.cs	Page Object y acciones de navegador.
Codigo	run-qa-tests.bat	Entrada de ejecucion para Windows con pausa final.
Codigo	run-qa-tests.ps1	Orquestacion completa y generacion de evidencias.
Documentacion	README.md	Guia de proyecto, pruebas, ejecucion y evidencias.
Documentacion	QA_Matriz_Evidencias_Pruebas.md	Trazabilidad y checklist de evidencias.
Documentacion	AUTOMATIZACION_PLAN.md	Plan por etapas y cobertura.

8. Analisis del proyecto

8.1 Fortalezas

- Separacion clara por capas y por proyectos de prueba.
- Cobertura API alineada con reglas de negocio criticas.
- Selenium visible permite evidencias demostrativas durante la entrega.
- El Page Object reduce fragilidad y centraliza selectores.
- El script automatico deja trazabilidad por ejecucion mediante logs y TRX.
- La documentacion conecta requisitos, casos, campos y evidencia.

8.2 Riesgos y consideraciones

- La persistencia InMemory es adecuada para entorno academico, pero no reemplaza pruebas contra base de datos real.
- Las capturas deben tomarse manualmente durante ejecucion visible si se requieren como evidencia grafica final.
- Selenium visible depende de Chrome/ChromeDriver disponible en el equipo.
- Para despliegue productivo se recomienda una plataforma compatible con ASP.NET Core, no Vercel directamente.

9. Conclusiones

El proyecto EFAC presenta una base tecnica consistente para pruebas y calidad de software. La automatizacion cubre escenarios funcionales clave de API y UI, incluyendo casos positivos, validaciones negativas, CRUD visual con Selenium y generacion de evidencias. La estructura documental y los scripts permiten reproducir el ciclo QA de forma clara, auditable y apta para entrega academica.

Anexo A. Comandos principales

Objetivo	Comando
Restaurar	dotnet restore Efac.sln
Compilar	dotnet build Efac.sln --no-restore
Pruebas API	dotnet test Efac.Tests.Api --no-build
Levantar WebAPI	dotnet run --project Efac.WebAPI --launch-profile http
Activar Selenium	\$env:EFAC_RUN_SELENIUM="true"
Base URL Selenium	\$env:EFAC_BASE_URL="http://localhost:5100/"
Chrome visible	\$env:EFAC_SELENIUM_HEADLESS="false"
Pruebas Selenium	dotnet test Efac.Tests.Selenium --no-build
Ciclo completo	.\run-qa-tests.bat

Anexo B. Resultado esperado de ejecucion

Resumen esperado

Restore: OK | Build: OK | Pruebas API: OK | Pruebas Selenium: OK | Suite completa: OK